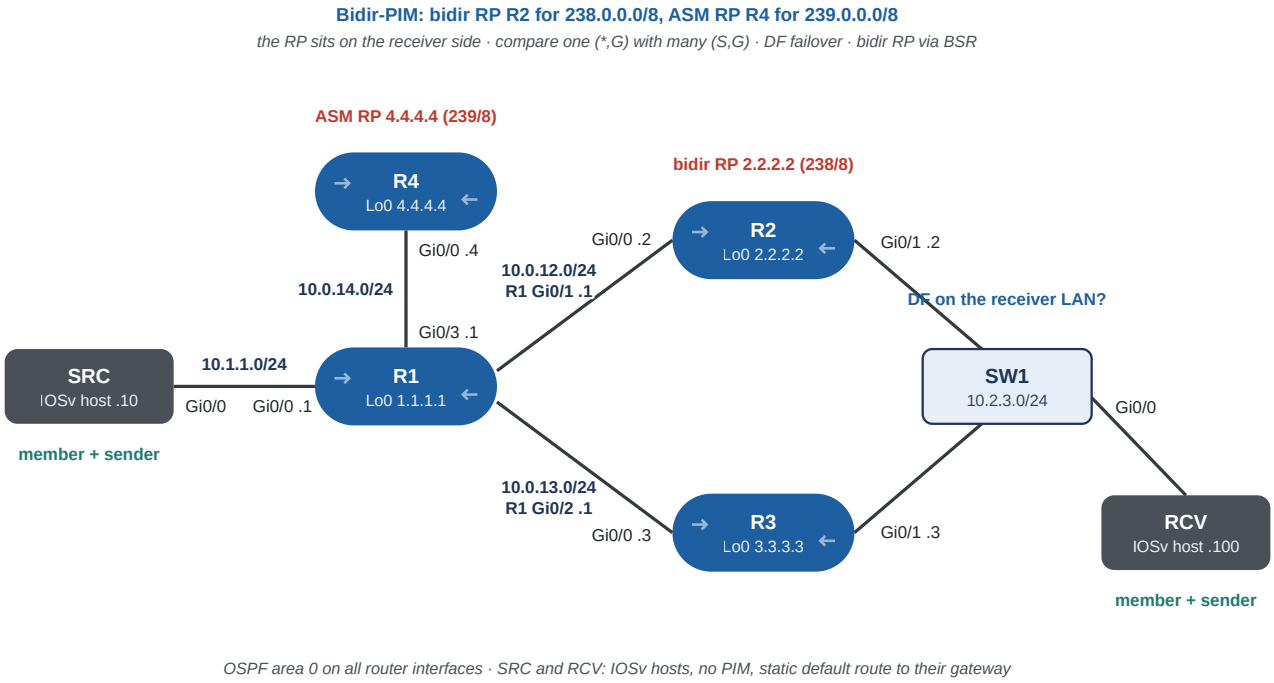
# PIM Lab 06: Multicast - Bidirectional PIM, Part 2

## Lab Objective:

The focus of this lab is to understand how Bidirectional PIM behaves next to classic PIM Sparse Mode in Cisco IOS routers. Additional technologies include separate ASM and bidir RPs, Designated Forwarder (DF) election with the RP on the receiver side, multicast state for many-to-many traffic, DF failover, and advertising a bidir RP with BSR.

## Lab Topology:

The lab network topology is illustrated below:




## IMPORTANT NOTE

Start from a fresh lab with only the addressing and OSPF from PIM Lab 01. No multicast configuration yet.

```
R1:  Gi0/0 10.1.1.1/24  Gi0/1 10.0.12.1/24  Gi0/2 10.0.13.1/24  Gi0/3 10.0.14.1/24
     Lo0 1.1.1.1/32
R2:  Gi0/0 10.0.12.2/24  Gi0/1 10.2.3.2/24  Lo0 2.2.2.2/32
R3:  Gi0/0 10.0.13.3/24  Gi0/1 10.2.3.3/24  Lo0 3.3.3.3/32
R4:  Gi0/0 10.0.14.4/24  Lo0 4.4.4.4/32
SRC: Gi0/0 10.1.1.10/24   default route to 10.1.1.1
RCV: Gi0/0 10.2.3.100/24  default route to 10.2.3.2
```

OSPF area 0 runs on every router interface, with manual router IDs (the Loopback0 address). The receiver LAN (Gi0/1 on R2 and R3) is passive in OSPF. SRC and RCV keep `ip routing` and use a static default route. They never run PIM.

Don't name an ACL `BIDIR`: IOS reads it as the `bidir` keyword and rejects the `rp-address` command as ambiguous.

## Task 1

Enable multicast routing and PIM Sparse Mode on all interfaces of R1, R2, R3 and R4, including Loopback0. Enable Bidir-PIM on all four routers. Configure static RPs on all four routers:

- R4's Loopback0 is a classic (ASM) RP for groups 239.0.0.0/8
- R2's Loopback0 is a bidirectional RP for groups 238.0.0.0/8

Verify the group-to-RP mapping on R3.

## Task 2

Before you look at any output, write on your topology drawing which router you expect to be the DF on each link for RP 2.2.2.2. Then verify the DFs on R1 and R3, and compare them with your prediction.

## Task 3

Configure RCV to join 238.1.1.1 and 239.1.1.1. SRC sends to each group in turn. Verify that RCV answers for both groups. Compare the multicast routing state for the two groups on R1, and find the path each stream takes from SRC to RCV. Does R4 receive the 238.1.1.1 stream?

## Task 4

Make SRC and R4 (on Loopback0) members of both groups too, so that SRC, RCV and R4 are all members. Let each of the three send a few pings to 238.1.1.1, then to 239.1.1.1. Compare how many multicast routing entries R1 holds for each group, and explain the difference.

## Task 5

Simulate a failure of R2's interface on the receiver LAN. Verify which router is now the DF on that LAN, and that RCV and SRC can still exchange 238.1.1.1 traffic. Restore the interface and verify the DF again.

## Task 6

Replace the static bidir RP with BSR: R1 is the candidate BSR, and R2 is a candidate RP for 238.0.0.0/8 in bidir mode. The ASM RP for 239.0.0.0/8 stays static. Verify that every router still treats 238.0.0.0/8 as bidir with RP 2.2.2.2. On how many routers did you have to configure the group range this time?

# Lab Validation:

The outputs below are trimmed. Timers, counters and uptimes will differ in your lab.

### Task 1

On **all** routers (R1 shown):

```
R1(config)#ip multicast-routing
R1(config)#ip pim bidir-enable
```

```
R1(config)#interface range GigabitEthernet0/0 - 3 , Loopback0
R1(config-if-range)#ip pim sparse-mode
R1(config-if-range)#exit
R1(config)#ip access-list standard ASM-GROUPS
R1(config-std-nacl)#permit 239.0.0.0 0.255.255.255
R1(config-std-nacl)#exit
R1(config)#ip access-list standard BIDIR-GROUPS
R1(config-std-nacl)#permit 238.0.0.0 0.255.255.255
R1(config-std-nacl)#exit
R1(config)#ip pim rp-address 4.4.4.4 ASM-GROUPS
R1(config)#ip pim rp-address 2.2.2.2 BIDIR-GROUPS bidir
```

On R2, R3 and R4, use the interface numbers they actually have (`interface range GigabitEthernet0/0 - 1 , Loopback0` on R2 and R3). Shutdown interfaces with PIM do no harm.

```
R3#show ip pim rp mapping
PIM Group-to-RP Mappings

Acl: BIDIR-GROUPS, Static, Bidir Mode
    RP: 2.2.2.2 (?)
Acl: ASM-GROUPS, Static
    RP: 4.4.4.4 (?)
```

One router can be an ASM RP and another a bidir RP in the same network. Each group range uses exactly one RP and one mode. Groups outside both ACLs have no RP and aren't forwarded.

## Task 2

```
R1#show ip pim interface df
* implies this system is the DF
Interface               RP              DF Winner        Metric     Uptime
GigabitEthernet0/0      2.2.2.2         *10.1.1.1        2          00:03:10
GigabitEthernet0/1      2.2.2.2          10.0.12.2       0          00:03:10
GigabitEthernet0/2      2.2.2.2         *10.0.13.1       2          00:03:10
GigabitEthernet0/3      2.2.2.2         *10.0.14.1       2          00:03:10

R3#show ip pim interface df
Interface               RP              DF Winner        Metric     Uptime
GigabitEthernet0/0      2.2.2.2          10.0.13.1       2          00:03:08
GigabitEthernet0/1      2.2.2.2          10.2.3.2        0          00:03:08
```

| Link | DF | Why |
|---|---|---|
| R1–R2 | R2 | R2 owns 2.2.2.2: metric 0 |
| Receiver LAN | R2 | Metric 0 beats R3's metric 3. No tie this time, so the IP address doesn't matter |
| R1–R3, R1–R4, SRC LAN | R1 | R1 reaches 2.2.2.2 with metric 2. R3 and R4 need 3 |

Each link elects the router closest to the RP. With the RP on R2, the picture is different from PIM Lab 05: R2 is DF on both of its links, and R4 is now just a leaf on the R1–R4 link.

## Task 3

```
RCV(config)#interface GigabitEthernet0/0
RCV(config-if)#ip igmp join-group 238.1.1.1
RCV(config-if)#ip igmp join-group 239.1.1.1
SRC#ping 238.1.1.1 repeat 5
Reply to request 0 from 10.2.3.100, 6 ms
```

```
SRC#ping 239.1.1.1 repeat 5
Reply to request 0 from 10.2.3.100, 7 ms
```

```
R1#show ip mroute 238.1.1.1
(*, 238.1.1.1), 00:01:12/00:02:47, RP 2.2.2.2, flags: B
  Bidir-Upstream: GigabitEthernet0/1, RPF nbr 10.0.12.2
  Outgoing interface list:
    GigabitEthernet0/1, Bidir-Upstream/Sparse, 00:01:12/stopped

R1#show ip mroute 239.1.1.1
(*, 239.1.1.1), 00:01:20/00:03:09, RP 4.4.4.4, flags: S
  Incoming interface: GigabitEthernet0/3, RPF nbr 10.0.14.4
  Outgoing interface list:
    GigabitEthernet0/2, Forward/Sparse, 00:01:20/00:03:09

(10.1.1.10, 239.1.1.1), 00:00:45/00:02:44, flags: FT
  Incoming interface: GigabitEthernet0/0, RPF nbr 0.0.0.0
  Outgoing interface list:
    GigabitEthernet0/2, Forward/Sparse, 00:00:45/00:02:44
```

The two streams take **different paths**:

- **238.1.1.1 (bidir):** SRC → R1 → **R2** → RCV. R1 is DF on the SRC LAN, so it sends the packets **up toward the RP** (Bidir-Upstream Gi0/1). R2 is the RP and the DF on the receiver LAN, so it delivers them to RCV. Only one (*,G), flag **B**, no Register.
- **239.1.1.1 (ASM):** SRC → R1 → **R3** → RCV. R3 is the **DR** (highest IP), so it joined. After the SPT switchover the stream goes directly from R1 to R3. R1 registered the source (flag **F**) with RP R4.

**R4 doesn't receive 238.1.1.1.** Bidir traffic always travels toward the RP and then down to members. R4 is neither on the way to R2 nor a member, so nothing is sent to it. In PIM Lab 05, R4 got every packet only because it *was* the RP.

## Task 4

```
SRC(config)#interface GigabitEthernet0/0
SRC(config-if)#ip igmp join-group 238.1.1.1
SRC(config-if)#ip igmp join-group 239.1.1.1
R4(config)#interface Loopback0
R4(config-if)#ip igmp join-group 238.1.1.1
R4(config-if)#ip igmp join-group 239.1.1.1
SRC#ping 238.1.1.1 repeat 5
RCV#ping 238.1.1.1 repeat 5
R4#ping 238.1.1.1 source Loopback0 repeat 5
```

Then the same three pings to 239.1.1.1. Each sender also gets a reply from itself, because it's a member of the group.

```
R1#show ip mroute 238.1.1.1 | include \(
(*, 238.1.1.1), 00:04:02/00:02:57, RP 2.2.2.2, flags: BC

R1#show ip mroute 239.1.1.1 | include \(
(*, 239.1.1.1), 00:04:10/00:03:19, RP 4.4.4.4, flags: SJC
(10.1.1.10, 239.1.1.1), 00:01:02/00:02:57, flags: FT
(4.4.4.4, 239.1.1.1), 00:00:41/00:02:18, flags: JT
(10.2.3.100, 239.1.1.1), 00:00:52/00:02:07, flags: JT
```

For the same three senders, R1 holds **1 entry** for the bidir group and **4 entries** for the ASM group: the (*,G) plus one (S,G) **per source**. With 500 senders, the ASM group would need 501 entries on every router in the

middle, and the bidir group would still need one. That's why bidir exists for many-to-many applications. The trade-off is that bidir traffic can't take a shortcut: it always goes through the RP link.

## Task 5

```
R2(config)#interface GigabitEthernet0/1
R2(config-if)#shutdown

R3#show ip pim interface GigabitEthernet0/1 df
Interface                RP               DF Winner        Metric     Uptime
GigabitEthernet0/1       2.2.2.2          *10.2.3.3        3          00:00:09

RCV#ping 238.1.1.1 repeat 5
Reply to request 0 from 10.2.3.100, 1 ms
Reply to request 0 from 10.1.1.10, 9 ms
```

R3 is now the only PIM router on the LAN, so it becomes the DF and handles RCV in both directions: RCV's packets go up through R3 → R1 → R2, and SRC's packets come down R1 → R3. Multicast recovers by itself.

Test from **RCV**, not from SRC. If SRC pings, RCV gets the packets but can't reply: replies are unicast, and RCV's default route still points to 10.2.3.2, which is down. Multicast redundancy comes from PIM; unicast redundancy needs an FHRP such as HSRP. To check the SRC → RCV direction without replies, let SRC ping and run `show ip mroute 238.1.1.1 count` twice on R3: the forwarded counter goes up.

```
R2(config-if)#no shutdown
R3#show ip pim interface GigabitEthernet0/1 df
GigabitEthernet0/1       2.2.2.2           10.2.3.2        0          00:00:05
```

When R2 comes back it offers a better metric (0), so it takes the DF role back. The DF election runs again whenever a better offer appears.

## Task 6

On **all** routers:

```
R1(config)#no ip pim rp-address 2.2.2.2 BIDIR-GROUPS bidir
```

On R1 and R2:

```
R1(config)#ip pim bsr-candidate Loopback0
R2(config)#ip pim rp-candidate Loopback0 group-list BIDIR-GROUPS bidir
```

```
R3#show ip pim rp mapping
Group(s) 238.0.0.0/8
  RP 2.2.2.2 (?), v2, bidir
    Info source: 1.1.1.1 (?), via bootstrap, priority 0, holdtime 150
         Uptime: 00:00:48, expires: 00:01:41
Acl: ASM-GROUPS, Static
    RP: 4.4.4.4 (?)
```

The bidir flag travels **inside the BSR advertisement**, so the group range and the mode only had to be configured on **one router**: R2. The other routers no longer need the BIDIR-GROUPS ACL at all (you can remove it). They still need `ip pim bidir-enable`, otherwise they ignore bidir RPs.

The static line was removed first for a reason: by default IOS **prefers dynamic RP information (BSR or Auto-RP) over a static RP** for the same groups. The static RP only wins with `ip pim rp-address <rp> <acl> override`.

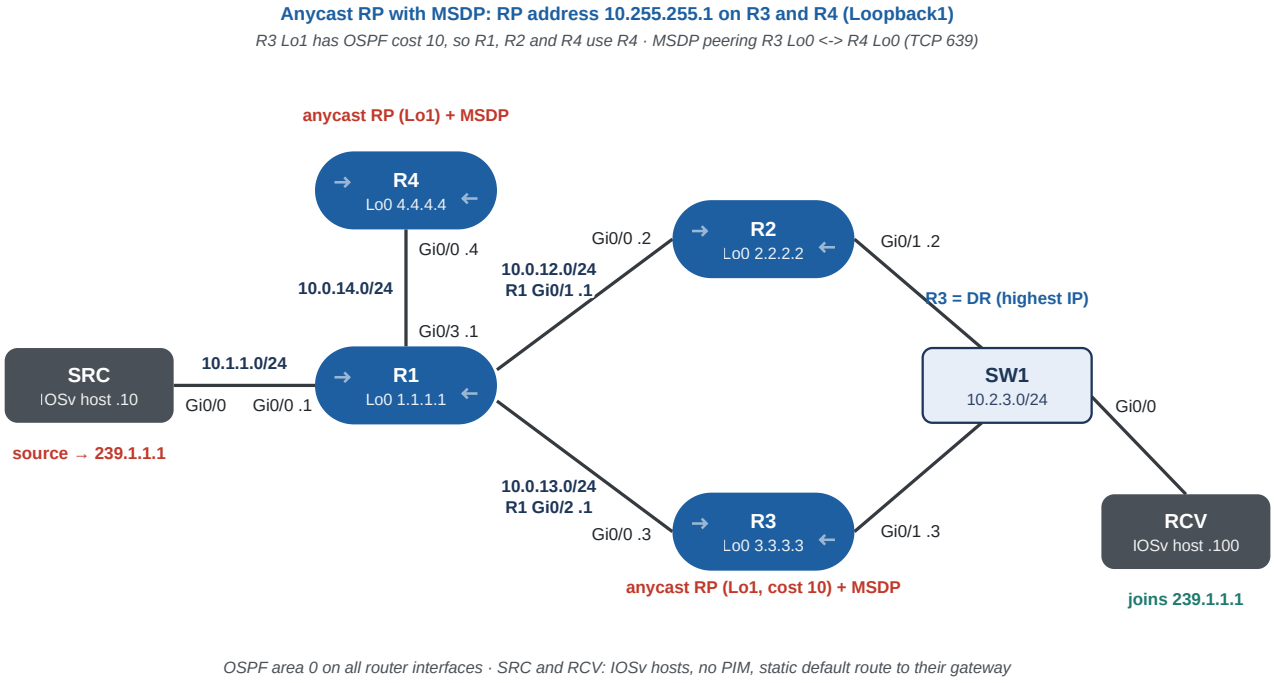
# PIM Lab 07: Multicast - Anycast RP with MSDP

## Lab Objective:

The focus of this lab is to understand Anycast RP with the Multicast Source Discovery Protocol (MSDP) in Cisco IOS routers. Additional technologies include the anycast RP address, IGP-based RP selection, MSDP peering and Source-Active (SA) messages, the MSDP originator ID, RP failover, and MSDP mesh groups.

## Lab Topology:

The lab network topology is illustrated below:




## IMPORTANT NOTE

Start from a fresh lab with only the addressing and OSPF from PIM Lab 01. No multicast configuration yet.

```
R1:  Gi0/0 10.1.1.1/24  Gi0/1 10.0.12.1/24  Gi0/2 10.0.13.1/24  Gi0/3 10.0.14.1/24
     Lo0 1.1.1.1/32
R2:  Gi0/0 10.0.12.2/24  Gi0/1 10.2.3.2/24  Lo0 2.2.2.2/32
R3:  Gi0/0 10.0.13.3/24  Gi0/1 10.2.3.3/24  Lo0 3.3.3.3/32
R4:  Gi0/0 10.0.14.4/24  Lo0 4.4.4.4/32
SRC: Gi0/0 10.1.1.10/24   default route to 10.1.1.1
RCV: Gi0/0 10.2.3.100/24  default route to 10.2.3.2
```

OSPF area 0 runs on every router interface. The receiver LAN (Gi0/1 on R2 and R3) is passive in OSPF. SRC and RCV keep `ip routing`, use a static default route and never run PIM.

The OSPF router IDs **must** be set by hand ( `router-id` = the Loopback0 address). In this lab several routers get the same Loopback1 address, and IOS would otherwise pick the highest loopback address as router ID, which gives duplicate router IDs.

## Task 1

Enable multicast routing and PIM Sparse Mode on all interfaces of R1, R2, R3 and R4, including Loopback0. Configure Loopback1 with 10.255.255.1/32 on **both** R3 and R4, with PIM Sparse Mode. Give R3's Loopback1 an OSPF cost of 10. Configure 10.255.255.1 as the static RP on all routers.

Verify which of the two RPs R1, R2, R3 and R4 each use, and explain why.

## Task 2

Configure RCV to join 239.1.1.1. SRC sends to 239.1.1.1. Verify that RCV does **not** answer. Examine the multicast routing state on R4 and R3, and explain what each RP knows and what it is missing.

## Task 3

Configure MSDP between R3 and R4 so that the two RPs share their sources. Use Loopback0 for the peering, and make sure that each RP identifies itself with its own unique address in the SA messages. Verify the MSDP peering, the SA cache on R3, and that RCV now answers SRC.

## Task 4

Simulate the failure of the primary RP by shutting down R4's Loopback1 while SRC keeps sending. Verify which RP R1 uses now and that RCV still answers. Restore Loopback1.

## Task 5

Add R2 as a third anycast RP: Loopback1 10.255.255.1/32 with an OSPF cost of 10 and PIM Sparse Mode. Build a full MSDP mesh between R2, R3 and R4, and configure it so that SAs received from one RP aren't re-sent to the other RPs. Verify the peerings and the SA cache on R2.

# Lab Validation:

The outputs below are trimmed. Timers, counters and uptimes will differ in your lab.

### Task 1

On **all** routers (R1 shown):

```
R1(config)#ip multicast-routing
R1(config)#interface range GigabitEthernet0/0 - 3 , Loopback0
R1(config-if-range)#ip pim sparse-mode
R1(config-if-range)#exit
R1(config)#ip pim rp-address 10.255.255.1
```

On R3 and R4:

```
R3(config)#interface Loopback1
R3(config-if)#ip address 10.255.255.1 255.255.255.255
R3(config-if)#ip pim sparse-mode
R3(config-if)#ip ospf cost 10
R4(config)#interface Loopback1
R4(config-if)#ip address 10.255.255.1 255.255.255.255
R4(config-if)#ip pim sparse-mode
```

```
R1#show ip route 10.255.255.1 | include entry|from|via
Routing entry for 10.255.255.1/32
  * 10.0.14.4, from 4.4.4.4, 00:01:31 ago, via GigabitEthernet0/3
R2#show ip route 10.255.255.1 | include entry|metric
Routing entry for 10.255.255.1/32
  Known via "ospf 1", distance 110, metric 3, type intra area
R3#show ip route 10.255.255.1 | include entry|connected
Routing entry for 10.255.255.1/32
  directly connected, via Loopback1
```

Every router has the **same RP address** in its config. Which physical RP it reaches is decided purely by **unicast routing**: the closest copy of 10.255.255.1 wins.

| Router | Uses RP | Why |
|---|---|---|
| R1, R2 | R4 | R4's copy costs 2 from R1 (3 from R2). R3's copy costs 11 (12) because of the cost 10 |
| R3 | R3 | Directly connected Loopback1 |
| R4 | R4 | Directly connected Loopback1 |

Without the cost 10, R1 would see two equal-cost copies of the RP. Anycast handles that fine, but the lab becomes hard to predict.

## Task 2

```
RCV(config)#interface GigabitEthernet0/0
RCV(config-if)#ip igmp join-group 239.1.1.1
SRC#ping 239.1.1.1 repeat 5
.....
```

```
R4#show ip mroute 239.1.1.1
(*, 239.1.1.1), 00:00:52/stopped, RP 10.255.255.1, flags: SP
  Incoming interface: Null, RPF nbr 0.0.0.0
  Outgoing interface list: Null

(10.1.1.10, 239.1.1.1), 00:00:52/00:02:07, flags: P
  Incoming interface: GigabitEthernet0/0, RPF nbr 10.0.14.1
  Outgoing interface list: Null

R3#show ip mroute 239.1.1.1
(*, 239.1.1.1), 00:03:40/00:02:21, RP 10.255.255.1, flags: SJC
  Incoming interface: Null, RPF nbr 0.0.0.0
  Outgoing interface list:
    GigabitEthernet0/1, Forward/Sparse, 00:03:40/00:02:21
```

The network is split in two:

- **R4 knows the source.** R1 (the first-hop router) registered SRC with its closest RP, R4. But R4 has **no receivers**: its outgoing list is Null, so it pruned the (S,G) (flag **P**) and sent a Register-Stop.
- **R3 knows the receiver.** R3 is the DR on the receiver LAN (highest IP), and its closest RP is itself, so the (*,G) join ended on R3. But R3 has **never heard of 10.1.1.10**: there's no (S,G), so it has nothing to join.

Both RPs work correctly, but each one only sees half of the picture. Anycast RP alone is broken. The RPs need a way to tell each other about their sources, and that's MSDP.

## Task 3

```
R3(config)#ip msdp peer 4.4.4.4 connect-source Loopback0
R3(config)#ip msdp originator-id Loopback0
R4(config)#ip msdp peer 3.3.3.3 connect-source Loopback0
R4(config)#ip msdp originator-id Loopback0
```

| Command | Meaning |
|---|---|
| `ip msdp peer 4.4.4.4 connect-source Loopback0` | TCP session (port 639) to the other RP's **unique** address, sourced from my own Loopback0 |
| `ip msdp originator-id Loopback0` | Put Lo0 (unique), not the shared anycast address, in the RP field of my SA messages |

The peering must use **Loopback0**, not Loopback1: 10.255.255.1 exists twice, so a TCP session to it would end on whichever copy is closest, possibly yourself. The **originator-id** matters for the same reason: an SA that says "RP 10.255.255.1" doesn't tell the other RPs *which* RP has the source, and the MSDP peer-RPF check on that address fails because the receiving RP owns it too.

```
R3#show ip msdp summary
MSDP Peer Status Summary
Peer Address     AS    State    Uptime/  Reset SA    Peer Name
                                Downtime Count Count
4.4.4.4          ?     Up       00:01:22 0     1     ?

R3#show ip msdp sa-cache
MSDP Source-Active Cache - 1 entries
(10.1.1.10, 239.1.1.1), RP 4.4.4.4, AS ?,00:00:58/00:05:31, Peer 4.4.4.4

SRC#ping 239.1.1.1 repeat 5
Reply to request 0 from 10.2.3.100, 9 ms

R3#show ip mroute 239.1.1.1 | begin \(10
(10.1.1.10, 239.1.1.1), 00:00:58/00:02:01, flags: MT
  Incoming interface: GigabitEthernet0/0, RPF nbr 10.0.13.1
  Outgoing interface list:
    GigabitEthernet0/1, Forward/Sparse, 00:00:58/00:02:01
```

What happened:

1. R4 had the active source from the Register, so it sent an **SA message** to R3: "source 10.1.1.10 sends to 239.1.1.1, originated by RP 4.4.4.4". It repeats it every 60 seconds while the source is active. If you configured MSDP after the source started, the SA can take up to a minute to arrive.
2. R3 has a (*,G) with a receiver, so it sent an **(S,G) join toward 10.1.1.10** via R1. The entry has flag **M** (MSDP created entry).
3. The stream now flows SRC → R1 → R3 → RCV. R4 is not on the path at all.

MSDP only exchanges **information about sources**. It never carries the multicast data itself.

## Task 4

```
R4(config)#interface Loopback1
R4(config-if)#shutdown

R1#show ip route 10.255.255.1 | include entry|metric|via
Routing entry for 10.255.255.1/32
  Known via "ospf 1", distance 110, metric 11, type intra area
```

```
  * 10.0.13.3, from 3.3.3.3, 00:00:07 ago, via GigabitEthernet0/2

SRC#ping 239.1.1.1 repeat 5
Reply to request 0 from 10.2.3.100, 6 ms
```

R4's copy of 10.255.255.1 disappeared from OSPF, so R1 and R2 now reach the RP address on R3. Nothing had to change in PIM or MSDP: the failover is just **IGP convergence**, a few seconds with OSPF. The running stream doesn't even notice, because it already flows on the source tree through R1 and R3. New sources now register directly with R3.

That's the main benefit of Anycast RP: **redundancy and load sharing with a static RP**, without Auto-RP or BSR.

```
R4(config-if)#no shutdown
```

## Task 5

On R2:

```
R2(config)#interface Loopback1
R2(config-if)#ip address 10.255.255.1 255.255.255.255
R2(config-if)#ip pim sparse-mode
R2(config-if)#ip ospf cost 10
R2(config-if)#exit
R2(config)#ip msdp peer 3.3.3.3 connect-source Loopback0
R2(config)#ip msdp peer 4.4.4.4 connect-source Loopback0
R2(config)#ip msdp originator-id Loopback0
R2(config)#ip msdp mesh-group ANYCAST-RP 3.3.3.3
R2(config)#ip msdp mesh-group ANYCAST-RP 4.4.4.4
```

On R3 and R4, add R2 as a peer and put all peers into the mesh group:

```
R3(config)#ip msdp peer 2.2.2.2 connect-source Loopback0
R3(config)#ip msdp mesh-group ANYCAST-RP 2.2.2.2
R3(config)#ip msdp mesh-group ANYCAST-RP 4.4.4.4
R4(config)#ip msdp peer 2.2.2.2 connect-source Loopback0
R4(config)#ip msdp mesh-group ANYCAST-RP 2.2.2.2
R4(config)#ip msdp mesh-group ANYCAST-RP 3.3.3.3
```

```
R4#show ip msdp summary
Peer Address     AS    State    Uptime/  Reset SA    Peer Name
                                Downtime Count Count
2.2.2.2          ?     Up       00:00:41 0     0     ?
3.3.3.3          ?     Up       00:25:10 0     0     ?

R2#show ip msdp sa-cache
MSDP Source-Active Cache - 1 entries
(10.1.1.10, 239.1.1.1), RP 4.4.4.4, AS ?,00:00:36/00:05:53, Peer 4.4.4.4
```

R2 learned the source **directly from R4**, the originator. In a full mesh every RP hears every SA first-hand, so forwarding SAs between peers would only create duplicates. The **mesh group** rule: an SA received from a member is **not forwarded to other members**, and the peer-RPF check is skipped for SAs from members. Every RP in an anycast set needs a peering with every other RP, so the number of peerings grows quickly (3 RPs need 3, 4 RPs need 6). In practice anycast sets are small, often just two RPs.

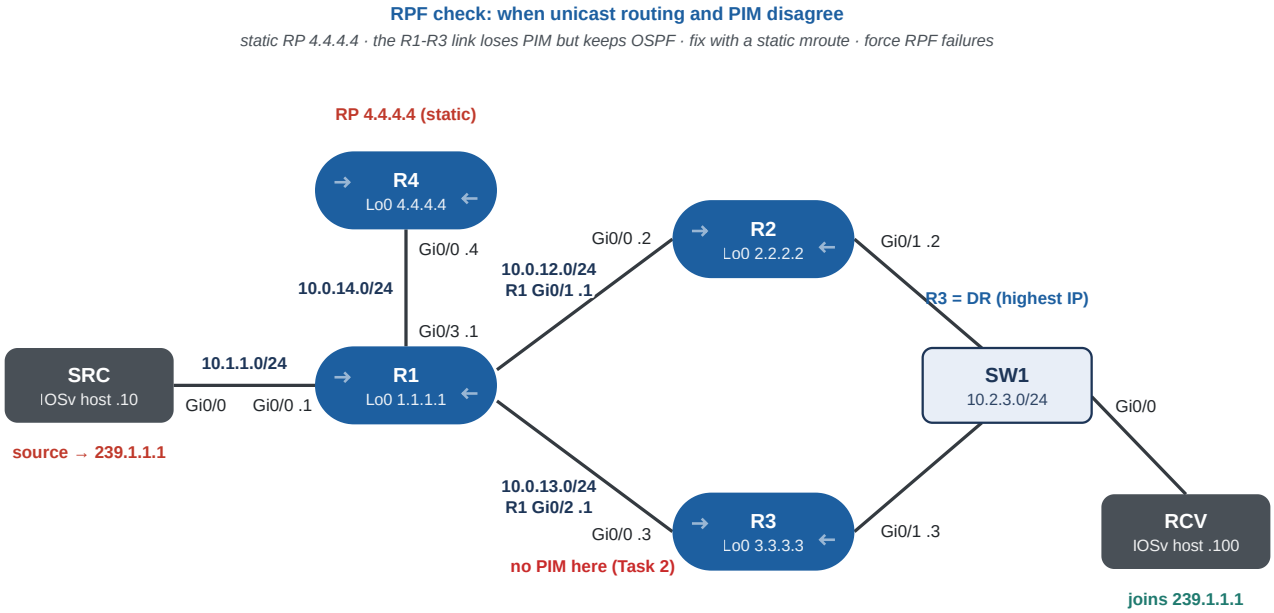
# PIM Lab 08: Multicast - RPF Check Failures

## Lab Objective:

The focus of this lab is to understand the Reverse Path Forwarding (RPF) check in Cisco IOS routers and how to troubleshoot it. Additional technologies include RPF lookups, a unicast path without PIM, static multicast routes ( `ip mroute` ), and RPF failure counters.

## Lab Topology:

The lab network topology is illustrated below:




## IMPORTANT NOTE

Start from a fresh lab with only the addressing and OSPF from PIM Lab 01. No multicast configuration yet.

```
R1:  Gi0/0 10.1.1.1/24  Gi0/1 10.0.12.1/24  Gi0/2 10.0.13.1/24  Gi0/3 10.0.14.1/24
     Lo0 1.1.1.1/32
R2:  Gi0/0 10.0.12.2/24  Gi0/1 10.2.3.2/24  Lo0 2.2.2.2/32
R3:  Gi0/0 10.0.13.3/24  Gi0/1 10.2.3.3/24  Lo0 3.3.3.3/32
R4:  Gi0/0 10.0.14.4/24  Lo0 4.4.4.4/32
SRC: Gi0/0 10.1.1.10/24   default route to 10.1.1.1
RCV: Gi0/0 10.2.3.100/24  default route to 10.2.3.2
```

OSPF area 0 runs on every router interface. The receiver LAN (Gi0/1 on R2 and R3) is passive in OSPF, so R3's only unicast path toward R1 is its link Gi0/0. SRC and RCV keep `ip routing` , use a static default route and never run PIM.

**The RPF check:** a router accepts a multicast packet only if it arrives on the interface the router itself would use to reach the **source** (or the **RP**, for the shared tree). That interface comes from the unicast routing table. PIM is "protocol independent" because it has no routing of its own: it trusts whatever the unicast table says.

## Task 1

Enable multicast routing and PIM Sparse Mode on all interfaces of R1, R2, R3 and R4, including Loopback0. Configure R4's Loopback0 as the static RP on all routers. Configure RCV to join 239.1.1.1, and verify that RCV answers SRC's pings to the group.

Verify the RPF information on R3 for the source and for the RP, and identify the RPF interface and the RPF neighbor.

## Task 2

Disable PIM on the link between R1 and R3, on both ends. Don't change OSPF. SRC sends to 239.1.1.1. Verify that RCV no longer answers, and troubleshoot why. Which router should be sending joins for RCV, and where would it send them?

## Task 3

Without enabling PIM on the R1–R3 link again, and without changing unicast routing, make the multicast stream reach RCV through R2. Configure only R3. Verify the new RPF information on R3, and the path the stream takes.

## Task 4

Enable PIM on the R1–R3 link again, but keep your configuration from Task 3 on R3. On R1, force the stream onto the R1–R3 link with a static IGMP join on Gi0/2. Verify on R3 what happens to the packets that arrive from R1, and explain why the router behaves that way.

## Task 5

Remove the static join from R1 and the static configuration from R3. Verify that R3 uses the R1–R3 link for RPF again.

# Lab Validation:

The outputs below are trimmed. Timers, counters and uptimes will differ in your lab.

### Task 1

On **all** routers (R1 shown):

```
R1(config)#ip multicast-routing
R1(config)#interface range GigabitEthernet0/0 - 3 , Loopback0
R1(config-if-range)#ip pim sparse-mode
R1(config-if-range)#exit
R1(config)#ip pim rp-address 4.4.4.4
RCV(config)#interface GigabitEthernet0/0
RCV(config-if)#ip igmp join-group 239.1.1.1
SRC#ping 239.1.1.1 repeat 5
Reply to request 0 from 10.2.3.100, 5 ms
```

```
R3#show ip rpf 10.1.1.10
RPF information for ? (10.1.1.10)
  RPF interface: GigabitEthernet0/0
```

```
  RPF neighbor: ? (10.0.13.1)
  RPF route/mask: 10.1.1.0/24
  RPF type: unicast (ospf 1)
  Doing distance-preferred lookups across tables

R3#show ip rpf 4.4.4.4
RPF information for ? (4.4.4.4)
  RPF interface: GigabitEthernet0/0
  RPF neighbor: ? (10.0.13.1)
  RPF route/mask: 4.4.4.4/32
  RPF type: unicast (ospf 1)
```

| Field | Meaning |
|---|---|
| RPF interface | The only interface on which R3 accepts packets from this source. Joins for it go out here |
| RPF neighbor | The upstream PIM router the joins are sent to |
| RPF type | Where the answer came from: **unicast (ospf 1)** here, **multicast (static)** after an `ip mroute` |

`show ip rpf` is the first command for any "the stream doesn't arrive" problem: it tells you the path PIM *wants* to use.

## Task 2

```
R1(config)#interface GigabitEthernet0/2
R1(config-if)#no ip pim sparse-mode
R3(config)#interface GigabitEthernet0/0
R3(config-if)#no ip pim sparse-mode
SRC#ping 239.1.1.1 repeat 5
.....
```

Troubleshoot from the receiver side up:

```
R3#show ip igmp groups 239.1.1.1
IGMP Connected Group Membership
Group Address    Interface                Uptime    Expires   Last Reporter
239.1.1.1        GigabitEthernet0/1       00:12:40  00:02:31  10.2.3.100

R3#show ip pim interface | include DR|0/1
Address          Interface                Ver/   Nbr    Query  DR         DR
10.2.3.3         GigabitEthernet0/1       v2/S   1      30     1          10.2.3.3

R3#show ip rpf 10.1.1.10 | include interface|neighbor
  RPF interface: GigabitEthernet0/0
  RPF neighbor: ? (10.0.13.1)

R3#show ip pim neighbor
Neighbor          Interface                Uptime/Expires    Ver   DR
Address                                                            Prio/Mode
10.2.3.2          GigabitEthernet0/1       00:14:02/00:01:31 v2    1 / S P G
```

Reading it in order:

1. The receiver is fine: R3 knows about the IGMP membership.
2. **R3 is the DR** on the LAN (highest IP), so R3 must send the joins.
3. R3's RPF interface toward the source (and the RP) is **Gi0/0**, because OSPF says so.
4. But R3 has **no PIM neighbor on Gi0/0**. Its only neighbor is R2, on the LAN. The join has nowhere to go.

Unicast and multicast now disagree: OSPF routes over a link that doesn't run PIM. R2 does have a working PIM path, but it isn't the DR, so it never joins. This is the classic RPF problem: the unicast best path isn't multicast-enabled. Depending on the IOS version, `show ip mroute 239.1.1.1` on R3 shows Gi0/0 or Null as the incoming interface. Either way, no traffic arrives.

## Task 3

```
R3(config)#ip mroute 10.1.1.0 255.255.255.0 10.2.3.2
R3(config)#ip mroute 4.4.4.4 255.255.255.255 10.2.3.2
```

`ip mroute <network> <mask> <rpf-neighbor>` is a **static multicast route**. It's used **only for the RPF check**, never to forward unicast. It tells R3: "for sources in 10.1.1.0/24, pretend the path goes via 10.2.3.2". A static mroute has administrative distance 0, so it wins over the OSPF route in the RPF lookup. You need one entry for the **source** (SPT) and one for the **RP** (shared tree).

```
R3#show ip rpf 10.1.1.10
RPF information for ? (10.1.1.10)
  RPF interface: GigabitEthernet0/1
  RPF neighbor: ? (10.2.3.2)
  RPF route/mask: 10.1.1.0/24
  RPF type: multicast (static)

SRC#ping 239.1.1.1 repeat 5
Reply to request 0 from 10.2.3.100, 7 ms

R2#show ip mroute 239.1.1.1 | begin \(10
(10.1.1.10, 239.1.1.1), 00:00:31/00:02:58, flags: T
  Incoming interface: GigabitEthernet0/0, RPF nbr 10.0.12.1
  Outgoing interface list:
    GigabitEthernet0/1, Forward/Sparse, 00:00:31/00:02:58
```

R3 (still the DR) now sends its joins out Gi0/1 **to R2**. R2 joins upstream via R1 and forwards the stream onto the receiver LAN. Path: **SRC → R1 → R2 → LAN**. Unicast from R3 still uses Gi0/0: only multicast takes the detour. This is how you steer multicast around a link that can't run PIM, for example a firewall, a non-Cisco device, or a tunnel.

## Task 4

```
R1(config)#interface GigabitEthernet0/2
R1(config-if)#ip pim sparse-mode
R1(config-if)#ip igmp static-group 239.1.1.1
R3(config)#interface GigabitEthernet0/0
R3(config-if)#ip pim sparse-mode
```

With SRC sending, run this twice on R3:

```
R3#show ip mroute 239.1.1.1 count
Group: 239.1.1.1, Source count: 1, Packets forwarded: 0, Packets received: 412
  Source: 10.1.1.10/32, Forwarding: 0/0/0/0, Other: 412/206/0
```

The three numbers after **Other** are: total packets received / **RPF failed** / other drops. The middle number grows with every packet R1 pushes onto Gi0/2.

R3 now gets every packet **twice**: once from R2 on Gi0/1 (its RPF interface, accepted) and once from R1 on Gi0/0 (not its RPF interface, **dropped**). R3 doesn't care that Gi0/0 is a perfectly good PIM link, or that R1 is

closer to the source. The static mroute says the source is behind Gi0/1, so anything from 10.1.1.10 arriving on another interface fails the RPF check.

That's exactly what the RPF check is for: in a network with redundant paths, the same packet can arrive on several interfaces. Accepting only the one on the path back to the source stops **duplicates and multicast loops**. The RPF failure here is caused by our configuration, not by a fault. In real networks, a growing RPF-failed counter means unicast routing and the multicast tree disagree. Look for asymmetric routing, a static mroute, or a non-PIM link in the path.

## Task 5

```
R1(config)#interface GigabitEthernet0/2
R1(config-if)#no ip igmp static-group 239.1.1.1
R3(config)#no ip mroute 10.1.1.0 255.255.255.0 10.2.3.2
R3(config)#no ip mroute 4.4.4.4 255.255.255.255 10.2.3.2

R3#show ip rpf 10.1.1.10 | include interface|type
  RPF interface: GigabitEthernet0/0
  RPF type: unicast (ospf 1)
```

The RPF type is back to **unicast (ospf 1)**, R3 joins directly toward R1 again, and R2 prunes the stream from the LAN.